

CSCI-4208: Developing Advanced Web Applications

Week 14: Lecture 27

React
Functional Components & Hooks

Overview: React Hooks

- What are Hooks
- state & useState
- state & useEffect

What are Hooks?

- Hooks provide the full capabilities of React in a more functional programming approach
- Hooks are an alternative to JavaScript classes to write fully fledged React Components
-

Why?

- It allows React to encapsulate properties of a function.
- The concept of a component that does not depend on anything else corresponds with the functional approach
 - i.e. The output being dependent on the inputs of the function
- The abstraction of a function can be elegantly used in this context

But why hooks?

- Functional components already did exist in React before hooks
- However they were limited in what they can do
 - They can accept properties and render HTML based on their properties
 - But they couldn't tie into some of the more fundamental and advanced features of React
 - Class based components could define special functions that had special properties that "hooked" into React functionality
- But with hooks, functions also have the same ability

Let's start with an example: useState

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={ () => setCount(count + 1) }>
        Click me
      </button>
    </div>
  );
}
```

Let's start with an example: useState

Demo Note:

to import useState, useEffect in codepen, use the following instead:

```
useState = React.useState;  
useEffect = React.useEffect;
```

useState example

- useState is a Hook
- the useState function returns two elements: a current state value and a function that enables you to update
- useState takes a single argument: the initial state value.

Now what really is an Hook?

- Hooks are functions that allow you "hook into" React features like state and what's called "lifecycle" features from function components
- An example of a life cycle feature is
- Execute code when component is first created
- Execute code when component updates

What is state in React?

- A state variable is a single piece of data that resides within each component
- Each instance of the component "remembers" its own state
- When any state variable is changed, React re-renders the component, incorporating any changes

Let's go back to our example

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

What's happening here?

- In our Example component, we set a single element of state called count
- We have access to the current value of count using the "count" variable, and the function "setCount" that takes one parameter (future state) that can change the count variable
- count, and setCount are declared as const to declaratively state that they cannot be changed. You cannot change count by assigning it to another value. But you must use the setCount function to change the count value
- Using the setCount function is important: when state is changed using this function React knows to render the component again after the variable has changed

What's happening here?

- Because the button's `onClick` attribute is set to an anonymous function that increments count (using the `setCount` function), the component is rendered again with the new value of the button

useEffect hook

- The useEffect hook gives you access to what's called React "lifecycle features"
- Lifecycle features in this case means access to special times in the creation, operation, and removal of a component.
- useEffect takes one function that will be executed right after the component is rendered to the screen.
- In effect, this hook gives you the ability to run code on startup, and when any state changes in the component, and when the component is removed

useEffect Example

```
useEffect(() => {  
  const subscription = props.source.subscribe();  
  return () => {  
    // Clean up the subscription  
    subscription.unsubscribe();  
  };  
});
```

useEffect Example

- `useEffect` here is passed a function that contains two statements:
- First, it is subscribing to whatever `props.source.subscribe()` is. This will be done any time this component is rendered to the screen
- Second, if this component is removed, then the function that is returned will execute (the unsubscribe action)
- This function it is returning enables you to clean up any actions that may not be needed anymore

Grand Example

```
import React, { useState, useEffect } from 'react';
import ReactDOM from 'react-dom';

function Weather(props) {

  const [temp, setTemp] = useState(0);

  let getWeatherData = async () => {
    let response = await fetch(`https://api.openweathermap.org/data/2.5/weather?q=${props.location}&appid=af578739923ac7f173a6054b24c606ea`);
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    else {
      return response.json();
    }
  }

  useEffect(() => {
    getWeatherData()
      .then( (response) => setTemp(response.main.temp) )
      .catch( e => console.log(e) );
  })

  return (
    <div>
      <strong>The temperature of {props.location} is {temp}</strong>
    </div>
  );
}
```

Grand Example

```
function App(props) {  
  return (  
    <div>  
      <Weather location="Berkeley,ca" />  
      <Weather location="Concord,ca" />  
    </div>  
  );  
}  
  
ReactDOM.render(  
  <App />,  
  document.getElementById('root')  
);
```

Grand Example

- Note we are using both `useState` to keep the state of the temperature and `useEffect` to make an API call to the weather endpoint to get the weather
- The function in `useEffect` is executed on every render, but since we only pass a property of the current location, it only needs to be rendered once

Grand Example

- Note we are using both `useState` to keep the state of the temperature and `useEffect` to make an API call to the weather endpoint to get the weather
- The function in `useEffect` is executed on every render, but since we only pass a property of the current location, it only needs to be rendered once

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

React Lab

Homework

Create a React App

CSCI-4208: Developing Advanced Web Applications

Week 14: Lecture 27

React
Functional Components & Custom Hooks

Overview: React Hooks II

- Rules of Hooks
- Custom Hooks

Rules of Hooks

- Must use hooks in functional React components
- Must use hooks at the top level of your components, or in your custom hooks
- This means don't use hooks inside conditionals either
- This is because React relies on the order of hooks to determine functionality
- If the order of hooks executed changes dynamically during execution, very hard to figure out bugs will appear

Custom Hooks

- React provides the ability to write your own hooks
- Custom Hooks provide another way to share stateful logic across components
- What is this stateful logic you speak of? Or what does that even mean?

Custom Hooks

- As your front end application becomes more complex, it becomes harder to manage all of the state variables as well as all of the logic that modifies those state variables
- What happens when you want to have some state that affects multiple components?
- React has provided a few ways of accomplishing this task, but typically involve creating more components that contain the shared state at a higher level.
- Hooks provide an alternative path that does not necessitate these "higher order components"

Setup Code

```
import React, { useState, useEffect } from 'react';

function FriendStatus(props) {
  const [isOnline, setIsOnline] = useState(null);

  useEffect( () => {
    const handleStatusChange = (status) => setIsOnline(status.isOnline);
    ChatAPI.subscribeToFriendStatus(props.friend.id, handleStatusChange);
    return () => ChatAPI.unsubscribeFromFriendStatus(props.friend.id, handleStatusChange);
  } );

  if (isOnline === null) {
    return 'Loading...';
  }

  return isOnline ? 'Online' : 'Offline';
}
```

Custom Hooks Example

- We have a component called FriendStatus that displays "Online" if the Friend was online and "Offline" if not
- Note the use of the useEffect hook to define what should happen when the component is created or is updated, and what should happen when it unsubscribes.

Custom Hooks Example

- However imagine if we had another component, a contact list, where we wanted to highlight a person's name if they were online.
- We would write out the logic, but it would require repeating a lot of the same code

Setup code 2

```
import React, { useState, useEffect } from 'react';

function FriendListItem(props) {
  const [isOnline, setIsOnline] = useState(null);

  useEffect(() => {
    const handleStatusChange = (status) => setIsOnline(status.isOnline);
    ChatAPI.subscribeToFriendStatus(props.friend.id, handleStatusChange);
    return () => ChatAPI.unsubscribeFromFriendStatus(props.friend.id, handleStatusChange);
  });

  return (
    <li style={{ color: isOnline ? 'green' : 'black' }}>
      {props.friend.name}
    </li>
  );
}
```

Refactoring?

- DRY Principle: put common code into a function and use the function instead
- Benefits include
 - not repeating code which can be error prone
 - once you update behavior for the shared function it is available everywhere
 - code is easier to read

Let's look at a custom hook

```
import { useState, useEffect } from 'react';

function useFriendStatus(friendID) {
  const [isOnline, setIsOnline] = useState(null);

  useEffect(() => {
    const handleStatusChange = (status) => setIsOnline(status.isOnline);
    ChatAPI.subscribeToFriendStatus(friendID, handleStatusChange);
    return () => ChatAPI.unsubscribeFromFriendStatus(friendID, handleStatusChange);
  });

  return isOnline;
}
```


What's happening here?

- The amazing thing is that a custom hook is also just another function: inputs, outputs, and logic
- You can use other hooks inside custom hooks
- You are writing stateful logic that can be shared across components
- You can also fully control the inputs and outputs of your hook. In this case, we pass in a friend ID and return whether that friend is online

How to use custom hook

```
function FriendStatus(props) {  
  const isOnline = useFriendStatus(props.friend.id);  
  
  if (isOnline === null) {  
    return 'Loading...';  
  }  
  return isOnline ? 'Online' : 'Offline';  
}
```

```
function FriendListItem(props) {  
  const isOnline = useFriendStatus(props.friend.id);  
  
  return (  
    <li style={{ color: isOnline ? 'green' : 'black' }}>  
      {props.friend.name}  
    </li>  
  );  
}
```

That was elegant

- Wasn't it? Those two components are now using the `useFriendStatus` custom hook, and thus reduced a lot of repeated code.
- If anything were to change with the API or the handling of the API results, we could simply update the `useFriendStatus` hook and all components using it would be updated

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

React Lab

Homework

Create a React App